

2026 年 CSP-J 第一套初赛模拟试题

建议用时：120 分钟 满分：100 分

姓名：_____ 班级：_____ 准考证号：_____ 座位号：_____

考生须知

1. 本卷共 42 题，分为单项选择题、阅读程序和完善程序三部分。
 2. 阅读程序中未列出选项的题目为判断题，正确填“√”，错误填“×”；其余题目均为单项选择题。
 3. 请将答案填写在卷末答题卡的相应位置。考试结束后，只交答题卡。
 4. 除题目特别说明外，程序均使用 C++ 语言编写，并假定输入数据合法。
-

一、单项选择题

1. 启动计算机引导操作系统是将操作系统（ ）。
 - A. 从磁盘调入中央处理器
 - B. 从内存存储器调入高速缓冲存储器
 - C. 从软盘调入硬盘
 - D. 从系统盘调入内存存储器
2. Windows 9x 是一种（ ）操作系统。
 - A. 单任务字符方式
 - B. 单任务图形方式
 - C. 多任务字符方式
 - D. 多任务图形方式
3. 在 24×24 点阵的字模中，汉字“一”与“编”的字模占用字节数分别是（ ）。
 - A. (72, 72)
 - B. (32, 32)
 - C. (32, 72)
 - D. (72, 32)
4. 计算机的运算速度取决于给定时间内其处理器所能处理的数据量。处理器一次能处理的数据量称为字长。已知 64 位的奔腾处理器一次能处理 64 位，相当于（ ）字节。
 - A. 8
 - B. 1
 - C. 16
 - D. 2

5. 算式 $(2047)_{10} - (3FF)_{16} + (2000)_8$ 的结果是 ()。
- A. $(2048)_{10}$
 - B. $(2049)_{10}$
 - C. $(3746)_8$
 - D. $(1AF7)_{16}$
6. 计算机的运算速度可以用 MIPS 来描述，它的含义是 ()。
- A. 每秒执行百万条指令
 - B. 每秒处理百万个字符
 - C. 每秒执行千万条指令
 - D. 每秒处理千万个字符
7. 设栈 S 的初始状态为空，现有 5 个元素组成的序列 $\{1, 2, 3, 4, 5\}$ ，对该序列在栈 S 上依次进行如下操作（从序列中的 1 开始，出栈后不再进栈）：进栈、出栈、进栈、进栈、出栈、进栈、出栈、进栈。出栈的元素序列是 ()。
- A. $\{5, 4, 3, 2, 1\}$
 - B. $\{2, 3\}$
 - C. $\{2, 3, 4\}$
 - D. $\{1, 3, 4\}$
8. 在有 n 个叶节点的哈夫曼树中，节点总数为 ()。
- A. 不确定
 - B. $2n - 1$
 - C. $2n + 1$
 - D. $2n$
9. 电线上停着两种鸟（A 和 B），可以看出相邻的两只鸟将电线划分为一个线段。这些线段可分为两类：一类是线段两端的鸟种类相同，另一类是线段两端的鸟种类不同。已知电线的两个端点处恰好停着种类相同的鸟，那么两端的鸟种类不同的线段数目一定是 ()。
- A. 奇数
 - B. 偶数
 - C. 可奇可偶
 - D. 数目固定
10. 从未排序序列中挑选元素，并将其依次放入已排序序列（初始时空）的一端，这种排序方法称 ()。
- A. 插入排序
 - B. 归并排序
 - C. 选择排序
 - D. 快速排序

11. 对于一棵满二叉树，若其叶节点数为 m 、分支节点数为 L 、总节点数为 n ，则下列关系式恒成立的是（ ）。
- A. $n = L + m$
 - B. $L + m = 2n$
 - C. $m = L - 1$
 - D. $n = 2L - 1$
12. 以下不是操作系统名字的是（ ）。
- A. Windows XP
 - B. Arch/Info
 - C. Linux
 - D. OS/2
13. 以下不是个人计算机的硬件组成部分的是（ ）。
- A. 主板
 - B. 虚拟内存
 - C. 总线
 - D. 硬盘
14. 已知元素 (8, 25, 14, 87, 51, 90, 6, 19, 20)，这些元素以（ ）的顺序全部入栈，再全部出栈，可使出栈顺序满足：8 在 51 之前；90 在 87 之后；20 在 14 之后；25 在 6 之前；19 在 90 之后。
- A. 20, 6, 8, 51, 90, 25, 14, 19, 87
 - B. 51, 6, 19, 20, 14, 8, 87, 90, 25
 - C. 19, 20, 90, 8, 6, 25, 51, 14, 87
 - D. 6, 25, 51, 8, 20, 19, 90, 87, 14
15. 假设我们用向量 $d = (a_1, a_2, \dots, a_5)$ 表示无向连通图 G 的 5 个顶点的度数，下面给出的哪一组 d 值合理？
- A. (2, 2, 2, 2, 2)
 - B. (1, 2, 2, 1, 1)
 - C. (3, 3, 3, 2, 2)
 - D. (5, 4, 3, 2, 1)

二、阅读程序

(1)

```
1 | #include <iostream>
2 | #include <cmath>
3 | using namespace std;
4 | bool IsPrime(int num) {
5 |     for (int i=2; i<=sqrt(num); i++) {
6 |         if (num % i == 0) return false;
7 |     }
8 |     return true;
9 | }
10 | int main() {
11 |     int num = 0;
12 |     cin >> num;
13 |     if (IsPrime(num)) cout << "YES" << endl;
14 |     else cout << "NO" << endl;
15 |     return 0;
16 | }
```

16. 输入 97 时，输出为 NO。
17. 输入 119 时，输出为 YES。
18. 若将第 5 行的 $\leq$ 改成 $<$ ，程序输出不会改变。
19. 当程序执行第 8 行时， i 的值为 $\sqrt{num}$ 。
20. 最坏情况下，此程序的时间复杂度是（ ）。
- A. $O(num)$
- B. $O(num^2)$
- C. $O(\sqrt{num})$
- D. $O(\log num)$
21. 若输入为 20 以内的正整数，则输出 YES 的概率是（ ）。
- A. 0.45
- B. 0.4
- C. 0.5
- D. 0.35

(2)

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | const int mod = 2048;
4 | long long c,n;
5 | long long kasumi (long long x, long long mi) {
6 |     long long res=1;
7 |     while (mi) {
8 |         if (mi & 1) {
9 |             res = (res * x) % mod;
10 |        }
11 |        x = (x * x) % mod;
12 |        mi >>= 1;
13 |    }
14 |    return res;
15 | }
16 | int main() {
17 |     cin >> n >> c;
18 |     if (n == 3) {
19 |         printf("%lld", c * (c - 1));
20 |         return 0;
21 |     }
22 |     long long ans = ((kasumi(c-1,n) + (c-1) * kasumi(-1,n)) % mod + mod) % mod;
23 |     cout << ans << endl;
24 |     return 0;
25 | }
```

22. 将第 9 行和第 11 行中的圆括号去掉，程序输出不变。
23. 将第 12 行的 `mi >>= 1` 改为 `mi *= 0.5`，程序输出不变。
24. 若输入 4 4，输出为 78。
25. 此程序的时间复杂度为 $O(\log n)$ 。
26. 若输入 3 4，输出为 ()。
- A. 8
- B. 12
- C. 18
- D. 19

(3)

```
1 | #include <stdio>
2 | int n, r, num[10000];
3 | bool mark [10000];
4 | void print() {
5 |     for (int i=1; i<=r; i++)
6 |         printf("%d ", num[i]);
7 |     printf("\n");
8 | }
9 | void search(int x) {
10 |     for (int i=1; i<=n; i++)
11 |         if (!mark[i]) {
12 |             num [x] = i;
13 |             mark[i] = true;
14 |             if (x == r) print();
15 |             search(x+ 1);
16 |             mark[i] = false;
17 |         }
18 | }
19 | int main() {
20 |     scanf ("%d%d", &n, &r);
21 |     search(1);
22 | }
```

27. 程序结束时，对任意 $1 \leq i \leq n$ ，都有 $\text{mark}[i] == 0$ 。

28. 若 $n < r$ ，则程序无输出。

29. 若输入 4 3，则输出中数字 1 和 2 的个数不同。

30. 此程序的时间复杂度为 $O(n)$ 。

31. 若输入 6 3，则函数 print 的执行次数为 ()。

A. 60

B. 120

C. 6

D. 720

32. 若输入 7 4，则输出的最后一行为 ()。

A. 4 5 6 7

B. 7 6 5 4

C. 4 3 2 1

D. 1 2 3 4

三、完善程序

(1)

Kruskal 求最小生成树的思想：首先将 n 个点看作 n 个独立的集合，将所有边按权值从小到大排序。然后按顺序枚举每一条边，判断它连接的两个点是否属于同一集合。若不属于同一集合，则将这条边加入最小生成树，并合并两个集合；否则跳过。直到找到 $n - 1$ 条边为止。

```

1 | #include <iostream>
2 | #include <algorithm>
3 | using namespace std;
4 | struct point { int x, y, v; } a[10000];
5 | int cmp(const point &a, const point &b) {
6 |     if(①) return 1;
7 |     return 0;
8 | }
9 | int fat [101];
10 | int father(int x) {
11 |     if (fat [x] != x) return fat[x] = ②;
12 |     return fat [x];
13 | }
14 | void unionn (int x, int y){
15 |     int fa = father(x), fb = father(y);
16 |     if (fa != fb) fat[fa] = fb;
17 | }
18 | int main() {
19 |     int i,j,n,m, k=0, ans=0, cnt=0;
20 |     cin >> n;
21 |     for (i=1; i<=n; i++)
22 |         for (j=1; j<=n; j++) {
23 |             cin >> m;
24 |             if (m != 0) {
25 |                 k++; a[k].x=i; a[k].y=j; a[k].v=m;
26 |             }
27 |         }
28 |     sort (a+1, a+1+k, ③);
29 |     for (i=1; i<=n; i++) fat[i] = i;
30 |     for (i=1; i<=k; i++){
31 |         if (father(a[i].x) != ④) {
32 |             ans += a[i].v;
33 |             unionn(a[i].x, a[i].y);
34 |             cnt++;
35 |         }
36 |         if (⑤) break;
37 |     }
38 |     cout << ans << endl;
39 |     return 0;
40 | }

```

33. ① 处应填 () 。

- A. $a.v < b.v$
- B. $a.v > b.v$
- C. $a.v \geq b.v$
- D. $a.v \leq b.v$

34. ② 处应填 () 。

- A. `father(x)`
- B. `father(fat[x])`
- C. `fat [father(x)]`
- D. `x`

35. ③ 处应填 ()。

- A. algorithm
- B. point
- C. cmp
- D. sizeof (a)

36. ④ 处应填 ()。

- A. a[i].y
- B. father(a[i].y)
- C. fat[a[i].y]
- D. a[i].x

37. ⑤ 处应填 ()。

- A. cnt>0
- B. i == 1
- C. ans == n-1
- D. cnt == n-1

(2)

欧拉路径问题是指从图中的一个顶点出发，是否能够一次性不回头地走遍所有的边（一次且仅一次）。算法代码如下。

```
1 | #include <iostream>
2 | using namespace std;
3 | int G[5][5];
4 | int visited [5][5];
5 | int n = 5;
6 | void euler (int u) {
7 |     for (int v=0; v<n; v++) {
8 |         if (G[u][v] && ①){
9 |             cout << u << "->" << v << endl;
10 |             visited[u][v] = visited[v][u] = ②;
11 |             ③
12 |         }
13 |     }
14 | }
15 | int main() {
16 |     G[1][2] = G[2][1] = G[1][3] = ④ = 1;
17 |     G[2][4] = G[4][2] = G[3][4] = ⑤ = 1;
18 |     euler (1);
19 |     return 0;
20 | }
```

38. ① 处应填 ()。

- A. G[v][u]
- B. !visited[u][v]
- C. visited[u][v]
- D. visited[v][u]

39. ② 处应填 () 。

A. 1

B. 0

C. u

D. v

40. ③ 处应填 () 。

A. euler(v);

B. euler(u);

C. G[u][v]=0;

D. G[v][u]=0;

41. ④ 处应填 () 。

A. G[0][1]

B. G[1][0]

C. G[3][1]

D. G[0][3]

42. ⑤ 处应填 () 。

A. G[0][2]

B. G[2][0]

C. G[2][1]

D. G[4][3]

答题卡

姓名：_____ 班级：_____ 准考证号：_____ 成绩：_____

判断题请填写“√”或“×”；其余题目请填写选项字母。

题号	1	2	3	4	5	6	7
答案	_____	_____	_____	_____	_____	_____	_____

题号	8	9	10	11	12	13	14
答案	_____	_____	_____	_____	_____	_____	_____

题号	15	16	17	18	19	20	21
答案	_____	_____	_____	_____	_____	_____	_____

题号	22	23	24	25	26	27	28
答案	_____	_____	_____	_____	_____	_____	_____

题号	29	30	31	32	33	34	35
答案	_____	_____	_____	_____	_____	_____	_____

题号	36	37	38	39	40	41	42
答案	_____	_____	_____	_____	_____	_____	_____